# Data Structures in C

CSC 230 : C and Software Tools

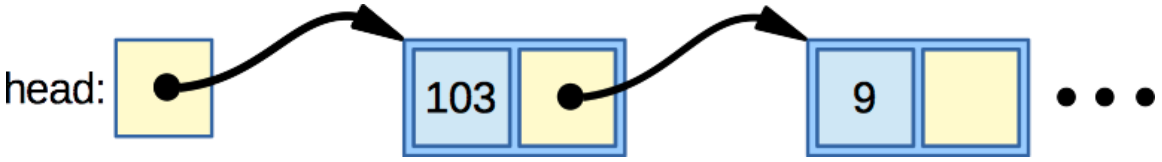
NC State Department of Computer Science

# Topics for Today

- Linked structures in C
- Linked lists
  - Methods for maintaining a list
  - Object-orientation, simulated
  - Deleting a list
  - Removing a node
- Binary search trees

# Linked Structures

- Two approaches to keeping data organized:
  - Pack values consecutively, in the same region of memory: an array or a struct.
  - Use pointers to make connections between different data items: a *linked structure*
- A linked list is a basic linked structure
  - With each node holding a value …
  - … and a pointer to the next node.

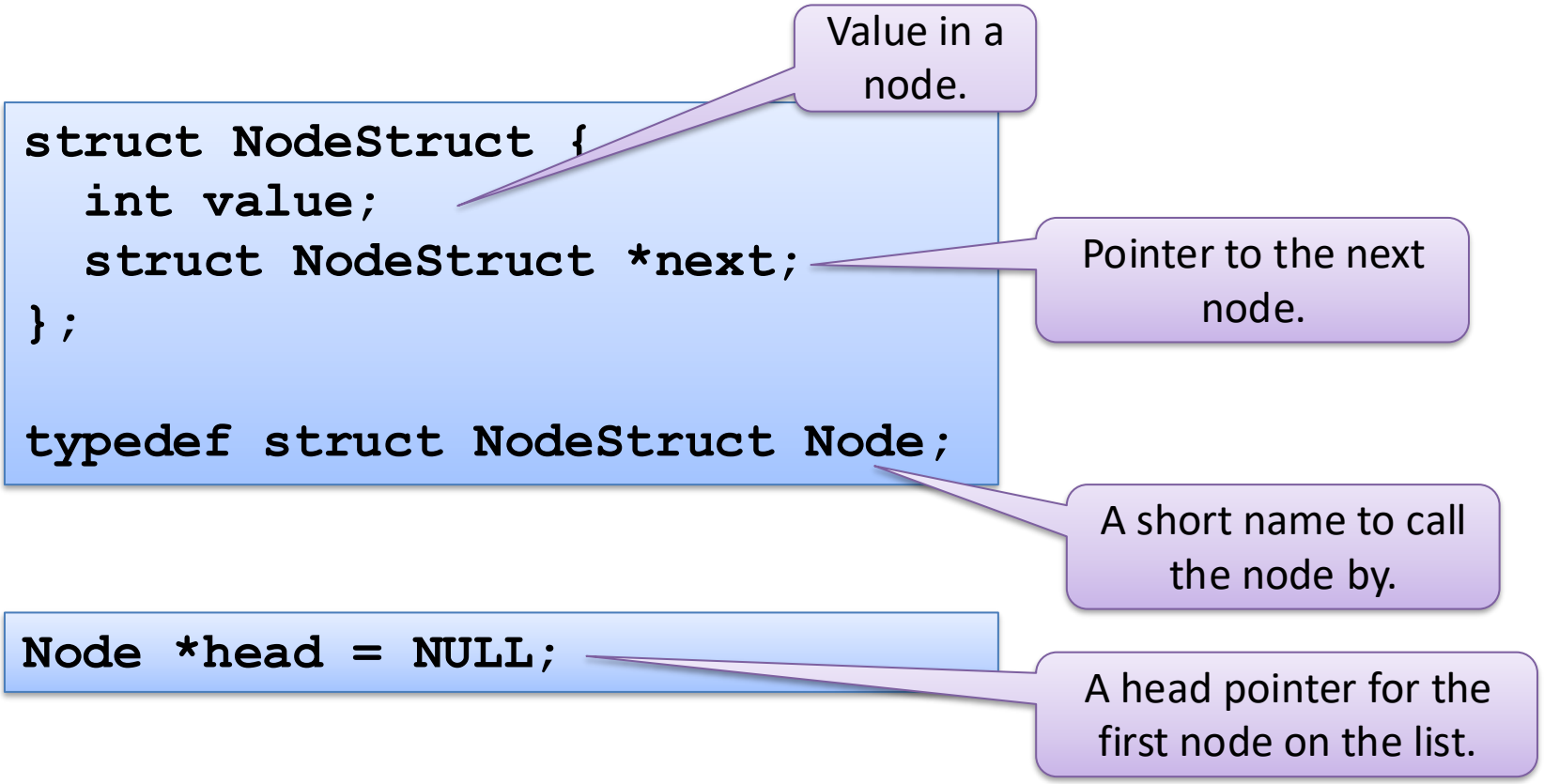


# Linked Structure Tradeoffs

- Compared to arrays, linked structures offer certain tradeoffs
  - Easier to grow the structure a little bit at a time ☺
  - Don't need to find large, contiguous regions of memory ☺
  - Can come up with some creative ways to organize the structure, well suited to certain operations ☺
    - Linked lists, balanced trees, disjoint sets, tries …
  - Locality may be worse, jumping all over memory ☹
  - Need to traverse links, instead of using pointer arithmetic to go right where we need to ☹

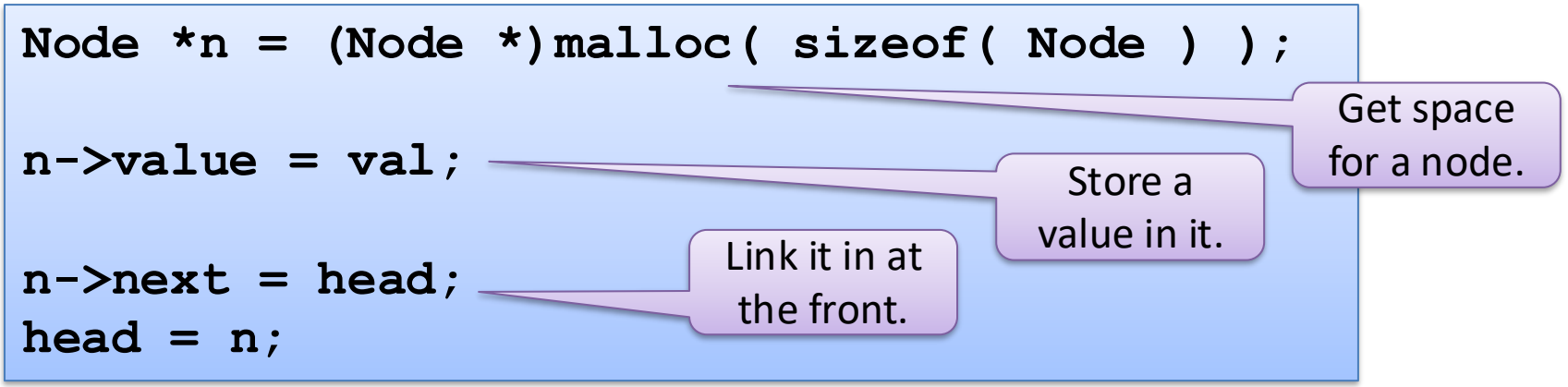
# Building a Linked List in C

- We can use a struct to represent a node.



# Adding to a List

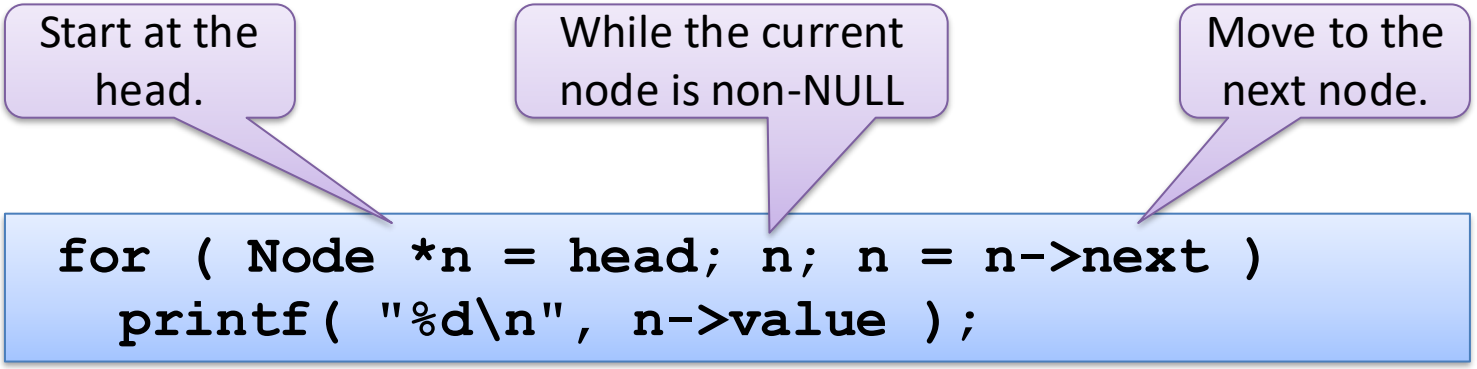
- We can use dynamic memory allocation to get space for each node.




- We could allocate a bunch of nodes statically
  - but dynamic allocation lets us grow the list as big as we need.

# Traversing a List

- In C, a NULL pointer evaluates to false
- We can use this to write some very simple traversal code



# Organizing our List Code

- We'd like to organize our code
  - Instead of putting linked list code all over the place
  - We'd like organize list operations into reusable functions
- For example:

```
Node *head = NULL;

void addValue( int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

  n->next = head;
  head = n;
}
```

Here's our list, a global pointer.

A function to add a new value.

Make a node holding the new value.

Link it into the list pointed to by head.

# Reusable Traversal Implementation

- We can use the *visitor* design pattern to make a reusable traversal function.

Pass me the function you want to call on each value.

```
void traverse( void (*visit)( int val ) )
{
  for ( Node *n = head; n; n = n->next )
    visit( n->value );
}
```

Call the visit function for each value.

# Using the Traverse Function

- Here's a visitor function to print each value.

```
static void myVisitor( int val )
{
  printf( "%d\n", val );
}
```

I print each value, without needing to know how the list is organized.

- Here's how to use the traversal function.

```
  traverse( myVisitor );
```

This gives the address of the myVisitor() function.

# Organizing our List Code

- This organization has some disadvantages.
  - We store the list as a global
  - So, we can only have one list, the global one.
- To support multiple, independent lists, we'll need to be more creative.

```
Node *head = NULL;

void addValue( int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

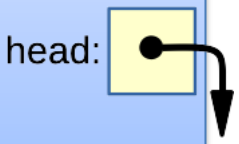
  n->next = head;
  head = n;
}
```

# Organizing our List Code

- How about this? We pass in the list as a parameter.

```
void addValue( Node *head, int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

  n->next = head;
  head = n;
}
```




```
  Node *myList = NULL;
  …;
  addValue( myList, 75 );
```

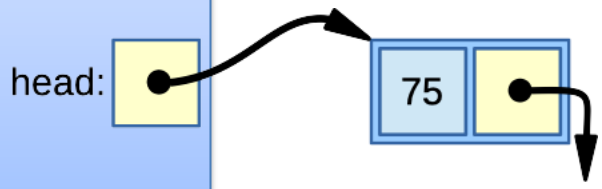


# Organizing our List Code

- But, a copy of the head pointer won't let you change the list.

```
void addValue( Node *head, int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

  n->next = head;
  head = n;
}
```




```
  Node *myList = NULL;
  …;
  addValue( myLi
```




See, the caller's head pointer is unchanged.

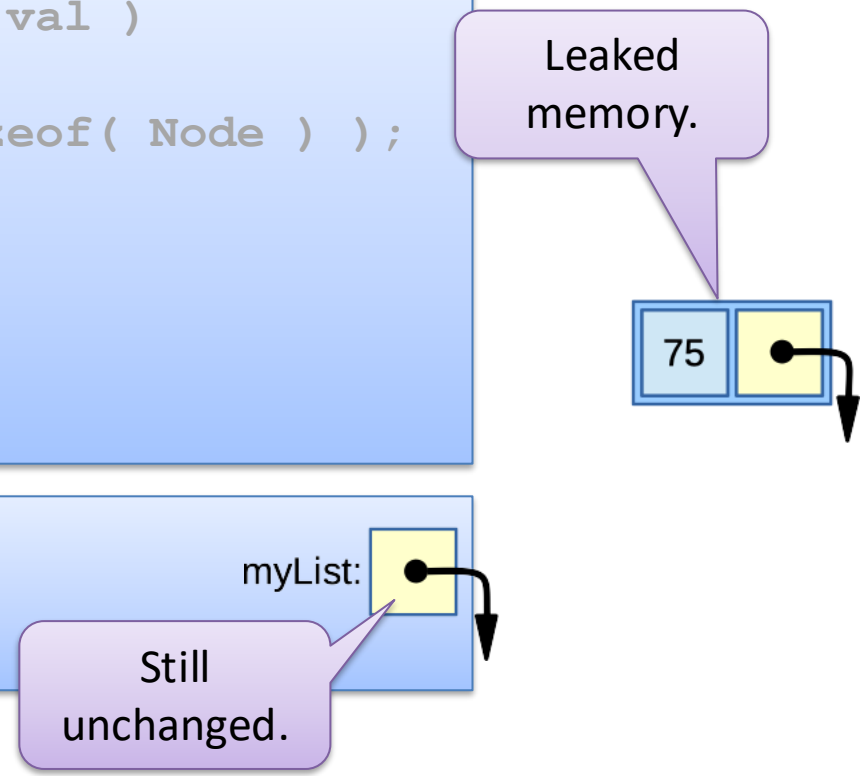
# Organizing our List Code

- When we return, the new node is unreachable.

```
void addValue( Node *head, int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

  n->next = head;
  head = n;
}
```

```
  Node *myList = NULL;
  …;
  addValue( myList, 75 );
```



# List Passing Alternatives

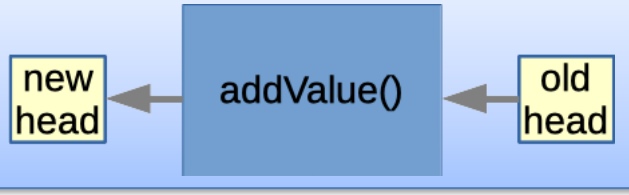
- One solution, return the new head pointer

```
Node *addValue( Node *head, int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

  n->next = head;
  return n;
}
```

Here's the new head of the list.

```
  Node *myList = NULL;
  …;
  myList = addValue( myList, 75 );
```




Be sure to update your head pointer.

# List Passing Alternatives

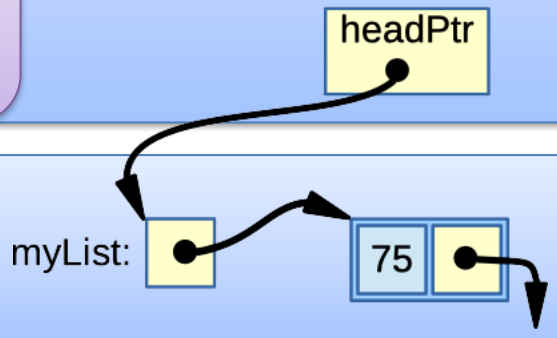
- Or, we could pass the head by reference, so the function could change it.
  - That's a pointer to a pointer to a Node

```
void addValue( Node **headPtr, int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

  n->next = *headPtr;
  *headPtr = n;
}
```

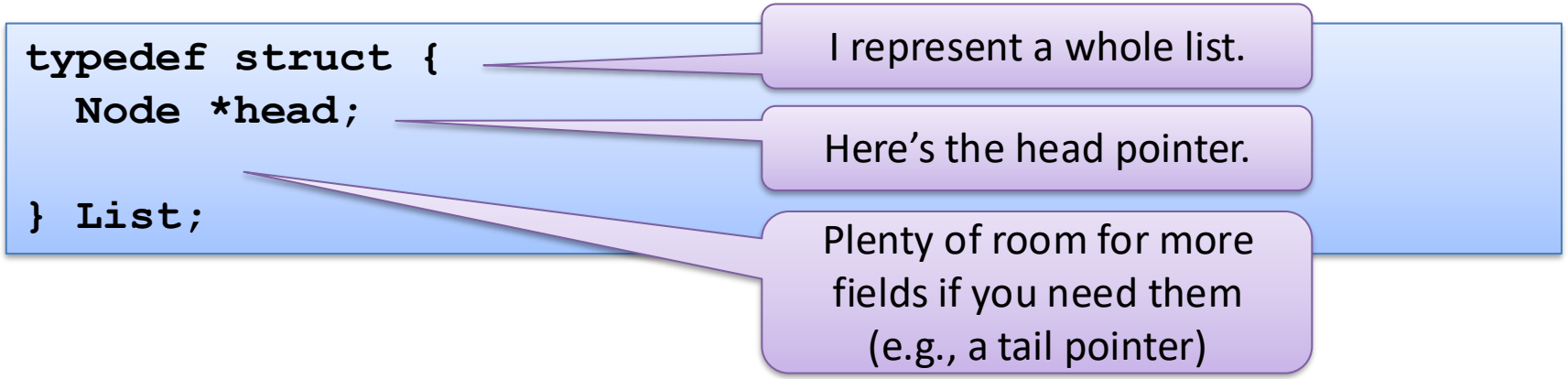
Pointer dereference to get to the caller's head pointer.

```
  Node *myList = NULL;
  …;
  addValue( &myList, 75 );
```



# List Passing Alternatives

- Or, more typically, we could make a little structure representing the list itself.



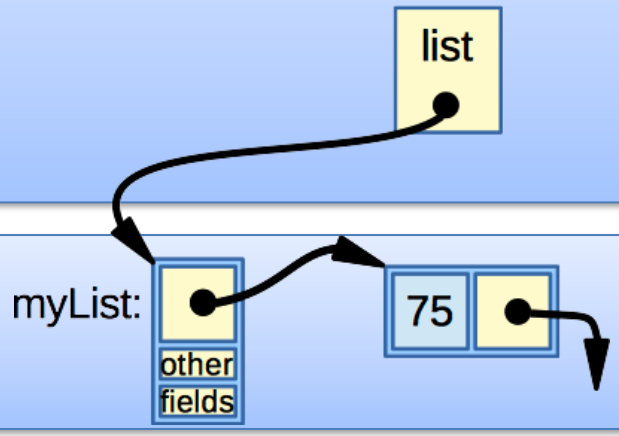

- We just need to pass a pointer to this struct to every function that uses a list.

# List Passing Alternatives

```
void addValue( List *list, int val )
{
  Node *n = (Node *)malloc( sizeof( Node ) );
  n->value = val;

  n->next = list->head;
  list->head = n;
}
```

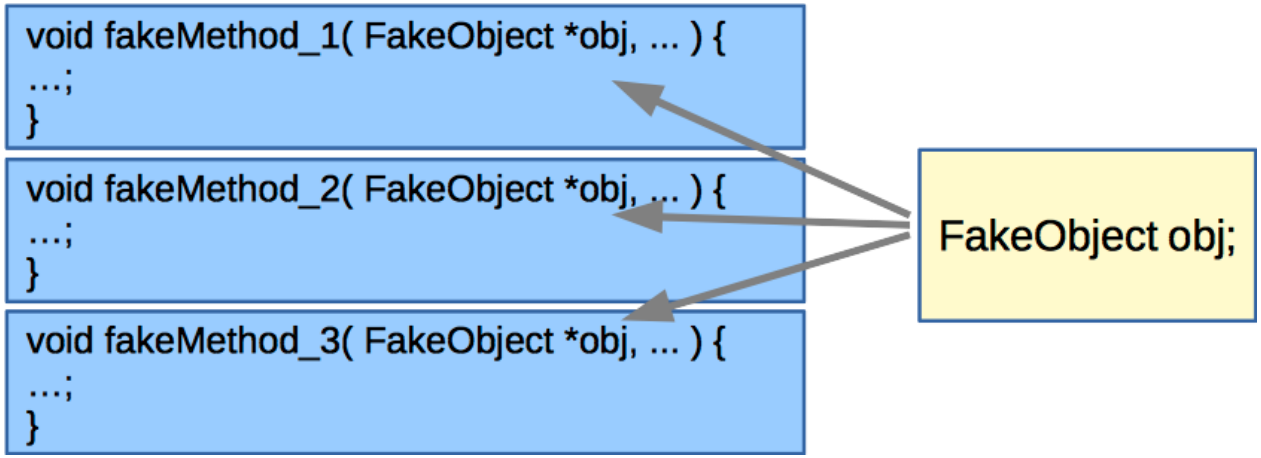
```
  List myList = { NULL };
  …;
  addValue( &myList, 75 );
```




- A pointer to a List struct lets a function:
  - Read fields related to the list
  - Or, modify the list if needed

# Faking Object Orientation

- We could use this to simulate object-orientation
- Instead of member functions we could
  - Define a bunch of (free) functions
  - Pass in a pointer to the object (struct) these functions are supposed to work on.




- Surprise! This is how Java and C++ work internally

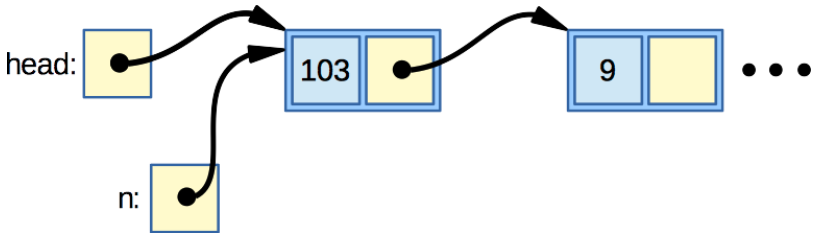
# Freeing Lists

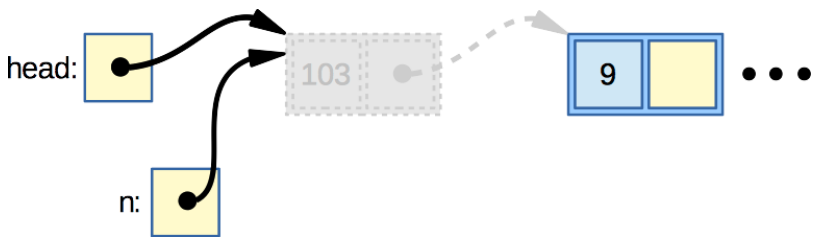
- Eventually, we have to free the memory for our linked list
- Here's a bad way to do it:

```
for ( Node *n = head; n; n = n->next )
  free( n );
```

Didn't you just free that node?

- This tries to use a node, right after we free it.

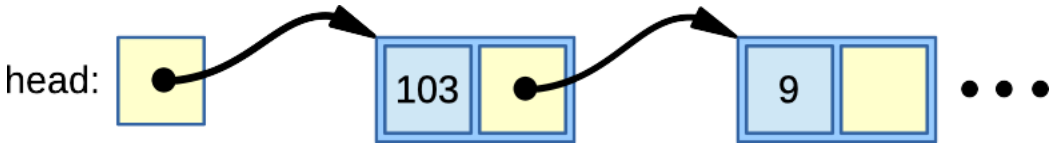


# Freeing Lists … Properly

- This is better:

```
while ( head ) {
  Node *next = head->next;
  free( head );
  head = next;
}
```

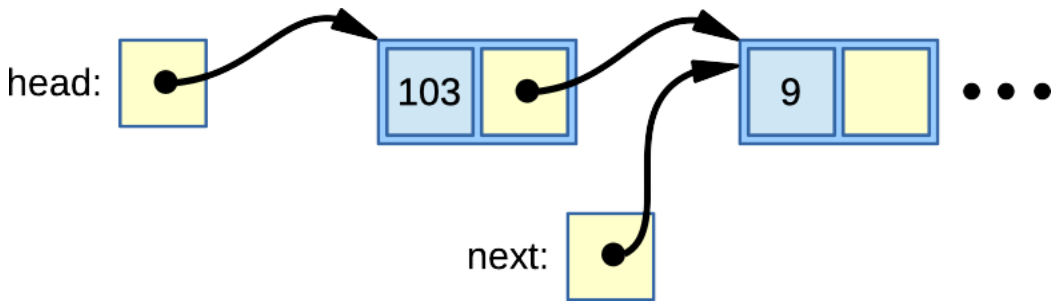


# Freeing Lists … Properly

- This is better:

```
while ( head ) {
  Node *next = head->next;
  free( head );
  head = next;
}
```

Copy the pointer to the next node.

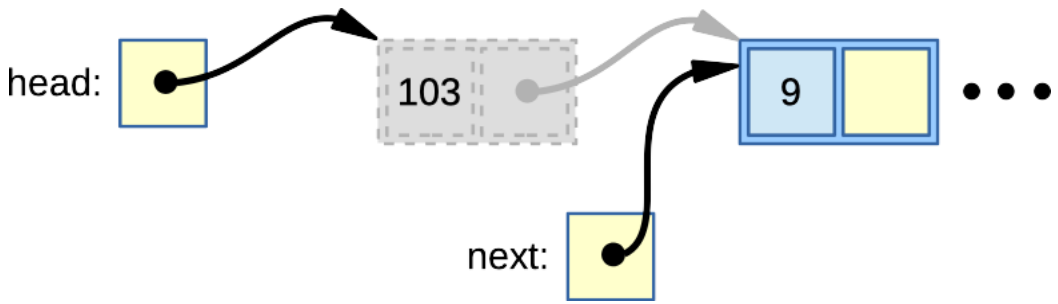


# Freeing Lists … Properly

- This is better:

```
while ( head ) {
  Node *next = head->next;
  free( head );
  head = next;
}
```

Free the old first node.

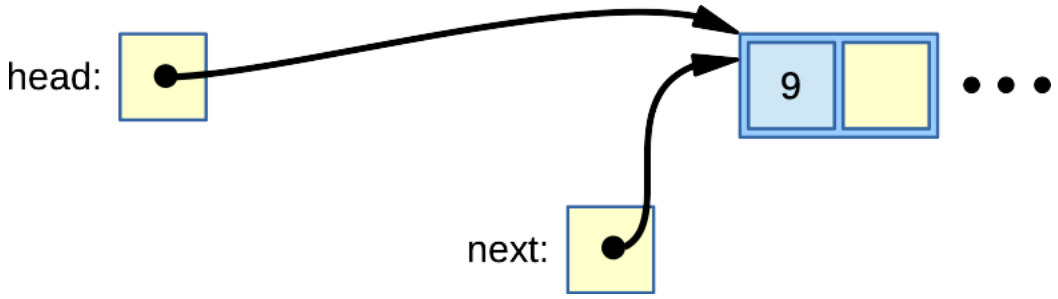


# Freeing Lists … Properly

- This is better:

```
while ( head ) {
  Node *next = head->next;
  free( head );
  head = next;
}
```

Move the head pointer to our copy of the next pointer.



# Removing Nodes

- To remove a node
  - We might choose to handle the head of the list as a special case
  - Then nodes later in the list if it's not the head node.

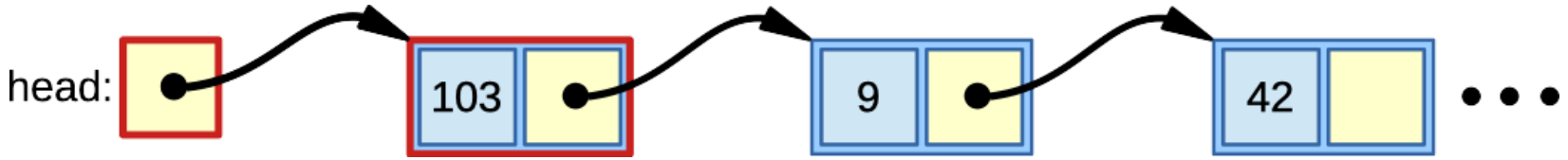
```
if ( head ) {
  if ( head->value == val ) {
    Node *n = head;
    head = head->next;

    free( n );
  }

  …
```

Is the target value in the first node?

If so, unlink and free it.



# Removing Nodes

```
  ...

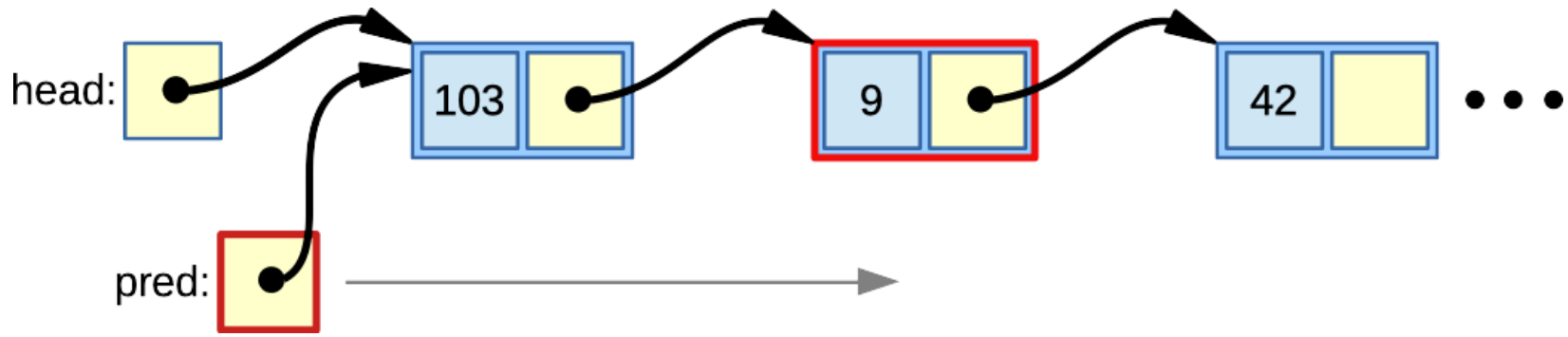
  Node *pred = head;
  while ( pred->next && pred->next->value != val )
    pred = pred->next;

  if ( pred->next ) {
    Node *n = pred->next;
    pred->next = pred->next->next;

    free( n );
  }
}
```

Look for the predecessor node, the one **before** the one you want to remove.

Unlink (and free) it.



# Getting Creative

- Access to pointers lets us be creative when we write code for linked structures
  - Having a pointer to something makes it possible to read or modify that thing.
  - … even another pointer.
- This comes in handy all over the place … if you know how to use it.
  - It lets us use the same code to manipulate the head pointer or a next pointer inside a node.

# An Idea that Almost Works

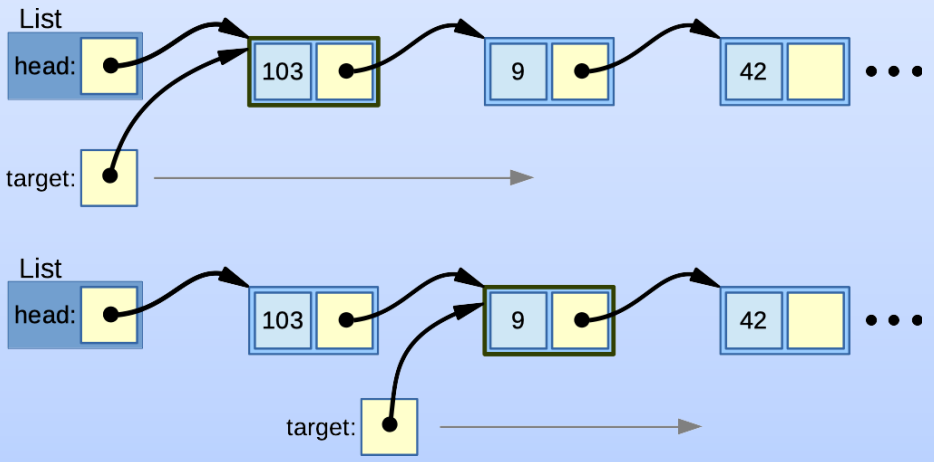
- How about this technique

```
bool removeValue( List *list, int val )
{
  Node *target = list->head;
  while (target &&
         target->value != val)
    target = target->next;

  if ( target ) {
    Node *n = target;
    target = target->next;

    free( n );
    return true;
  }


  return false;
}
```



# An Idea that Almost Works

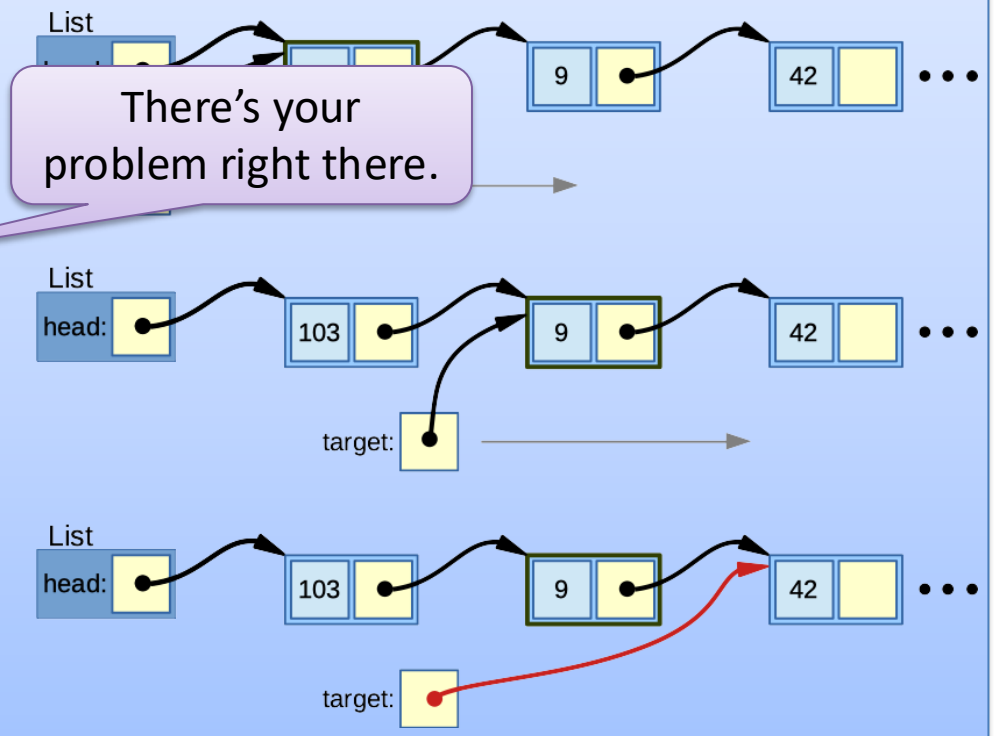
- How about this technique? (it doesn't work)

```
bool removeValue( List *list, int val )
{
  Node *target = list->head;
  while (target &&
         target->value != val)
    target = target->next;

  if ( target ) {
    Node *n = target;
    target = target->next;

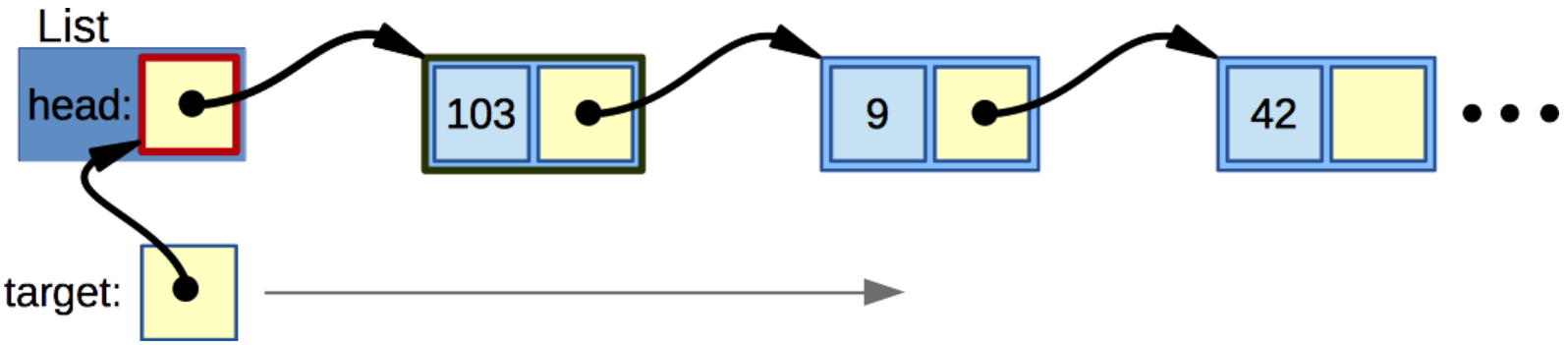
    free( n );
    return true;
  }


  return false;
}
```



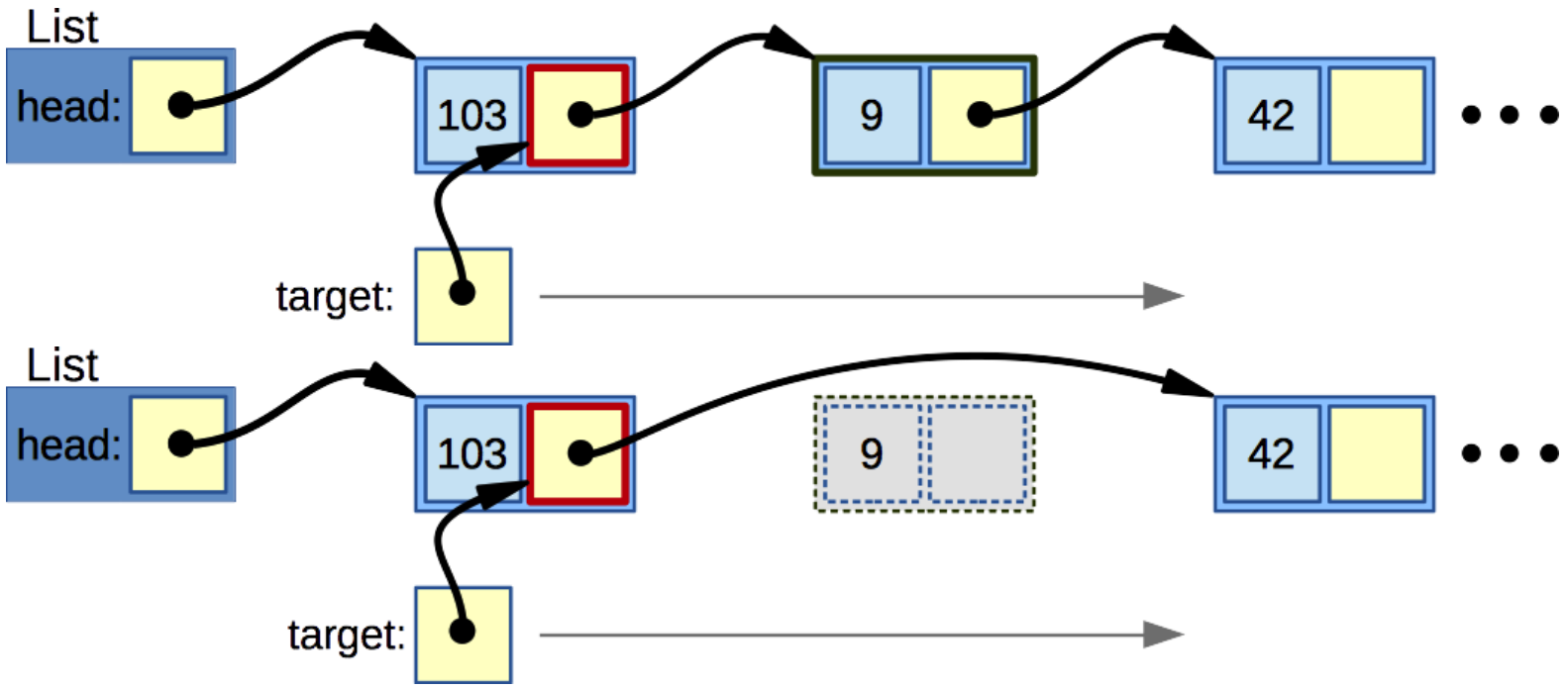
# Removing Nodes, “Simplified”

- To remove a node, we must change the pointer that points to it.
  - Every node has one such pointer
  - Either **the pointer inside its predecessor**.
  - **Or, the head pointer** itself.
- Both of these pointer have the same type (pointer to Node)
- We can handle them uniformly via a pointer to pointer to Node.
- We’ll start with a pointer pointing to the head pointer:



# Removing Nodes, “Simplified”

- As we traverse the list, we will move the ahead to pointers within the nodes.
  - This will give us a way to remove the next node on the list.



# Broken Code

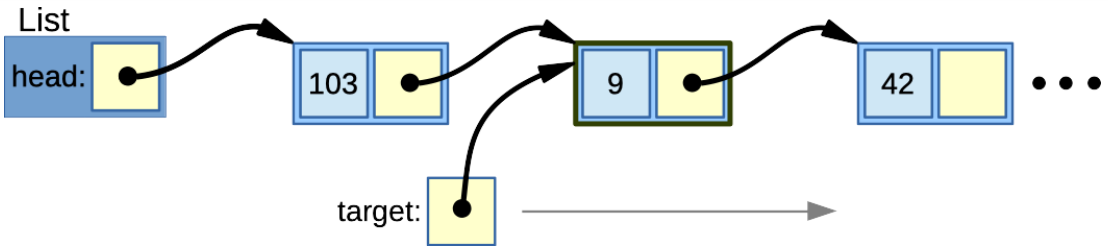
```
bool removeValue( List *list, int val )
{
  Node *target = list->head;
  while ( target && target->value != val )
    target = target->next;

  if ( target ) {
    Node *n = target;
    target = target->next;

    free( n );
    return true;
  }

  return false;
}
```

Here, we need to change the pointer inside the linked list.



# Removing Nodes, Simplified

```
bool removeValue( List *list, int val )
{
  Node **target = &list->head;
  while ( *target && (*target)->value != val )
    target = &(*target)->next;

  if ( *target ) {
    Node *n = *target;
    *target = (*target)->next;

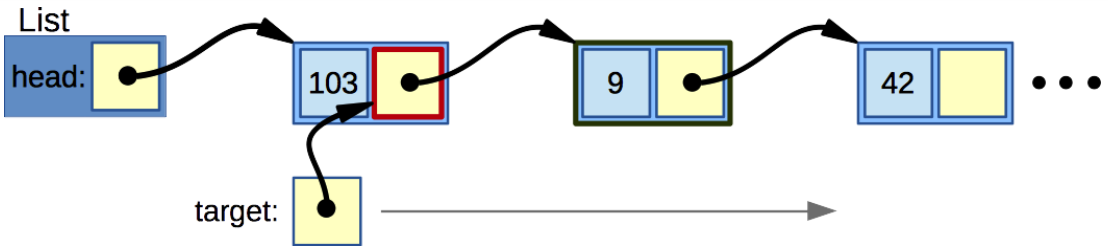
    free( n );
    return true;
  }

  return false;
}
```

I'm the address of the pointer to the node you're thinking about removing.

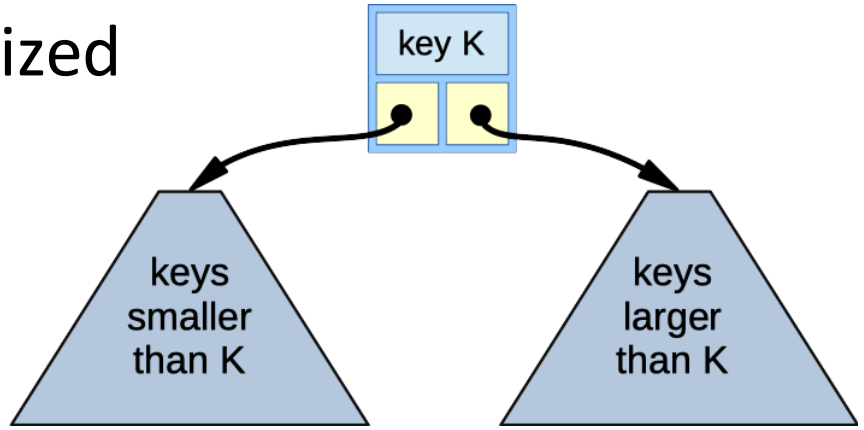
Start at the head, then walk down through all the pointers inside the nodes.

If we find a node containing val, remove its node by changing the pointer that points to it.



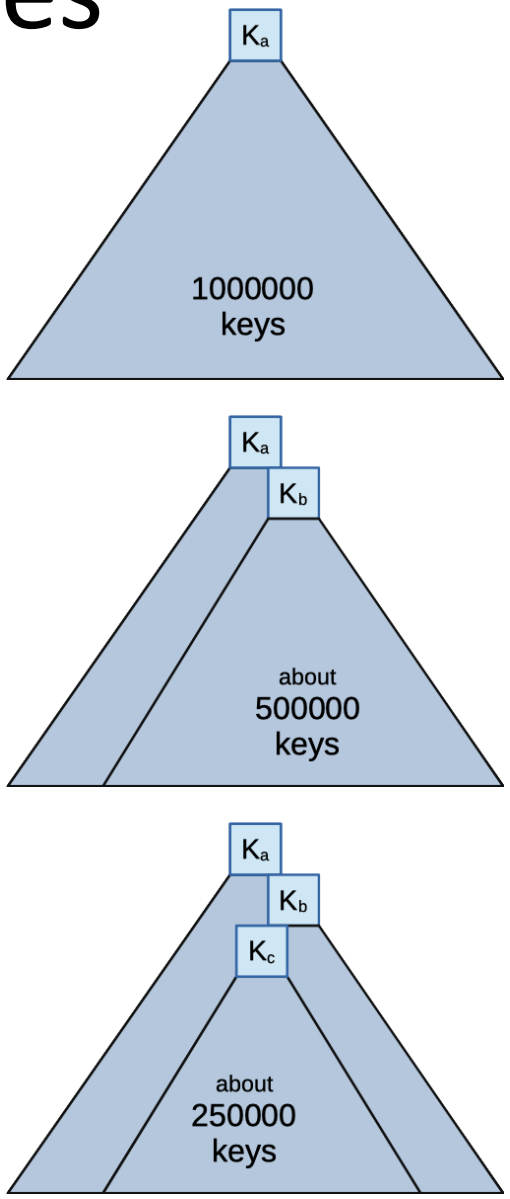
# Binary Search Trees

- A linked list is good for some things
  - A stack, a queue or a short list.
- Alternative organizations can give better performance for other operations.
- Like a Binary Search Tree
  - Every node has two pointers
  - Values (keys) are organized based on comparison



# Binary Search Trees

- Each comparison splits the set of values into two smaller subsets.
  - Ideally, each about the same size.
- Each time we traverse a node
  - we eliminate about half the keys.



# Building a BST

- We can represent the nodes of a BST as structs.

```
struct NodeStruct {
  int key;
  struct NodeStruct *left, *right;
};

typedef struct NodeStruct Node;
```

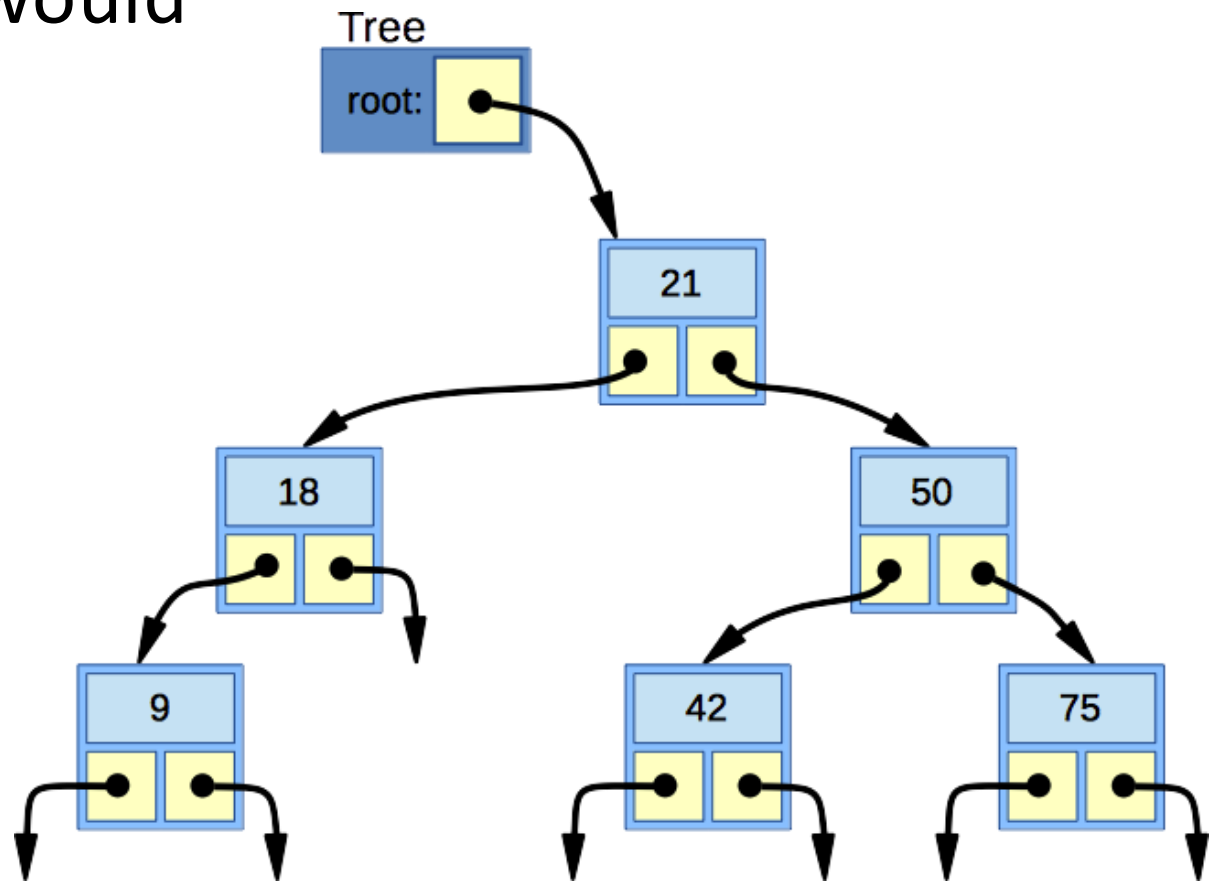
Short name for the Node structure

- And, we can have a struct for the whole tree.

```
typedef struct {
  Node *root;
} Tree;
```

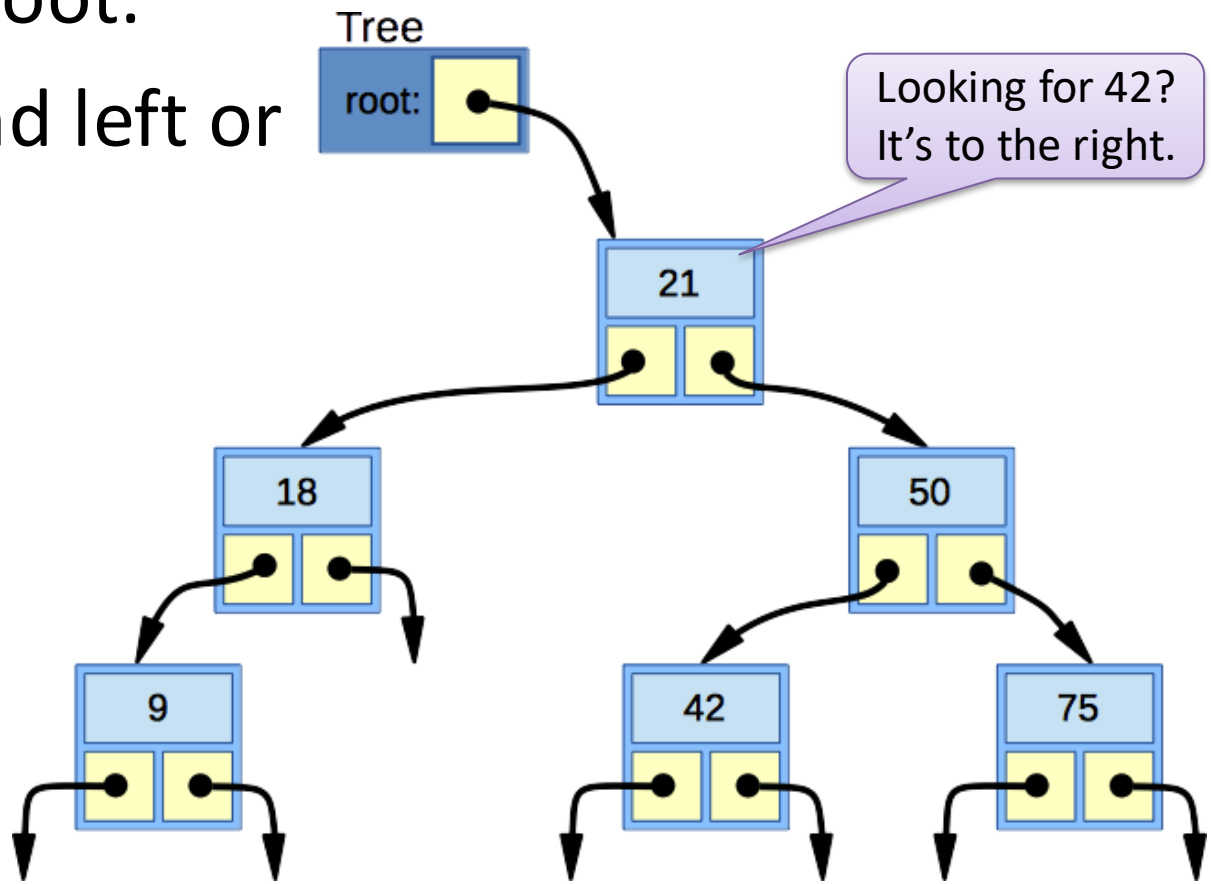
# Example Tree

- Here's what a typical (small) BST would look like:



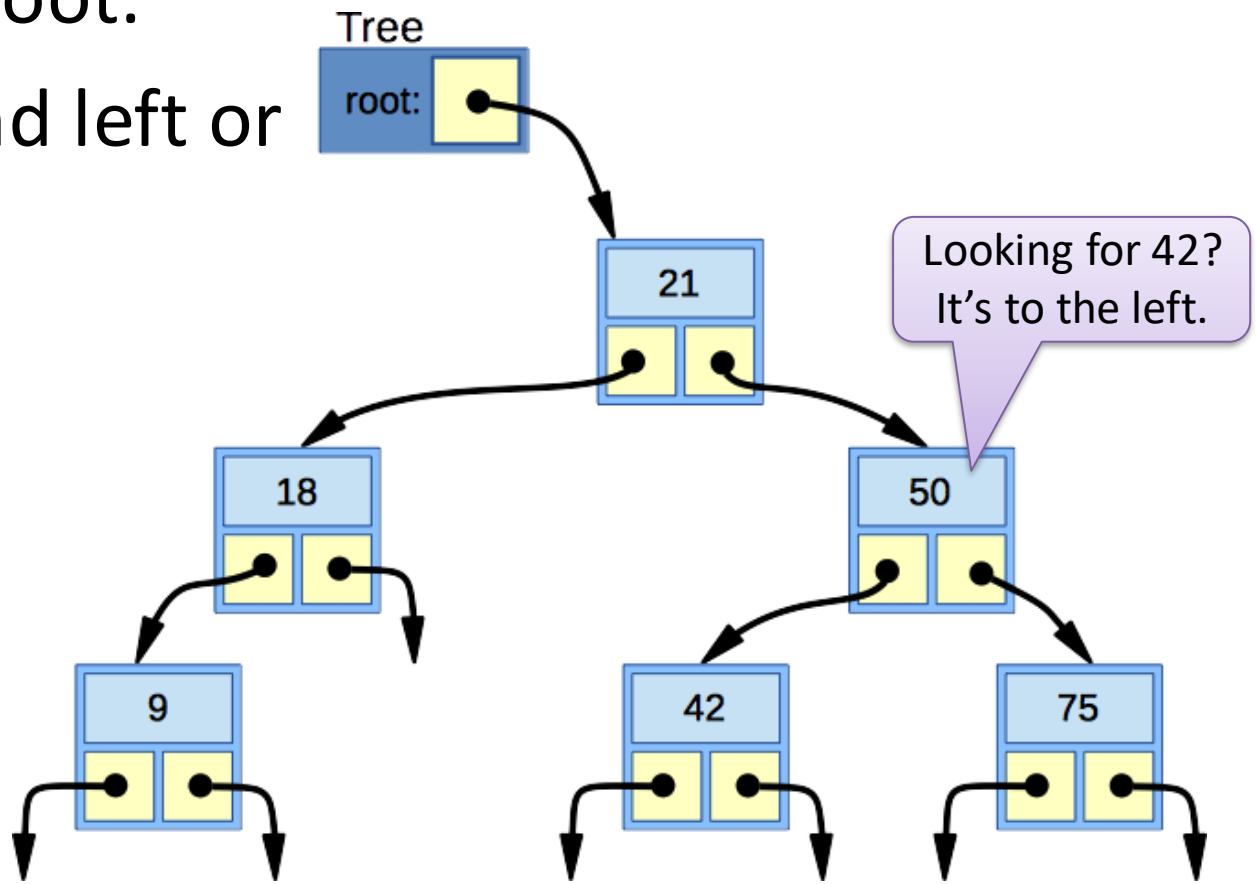
# Finding a Value

- To find a value, you start at the root.
- Then descend left or right.



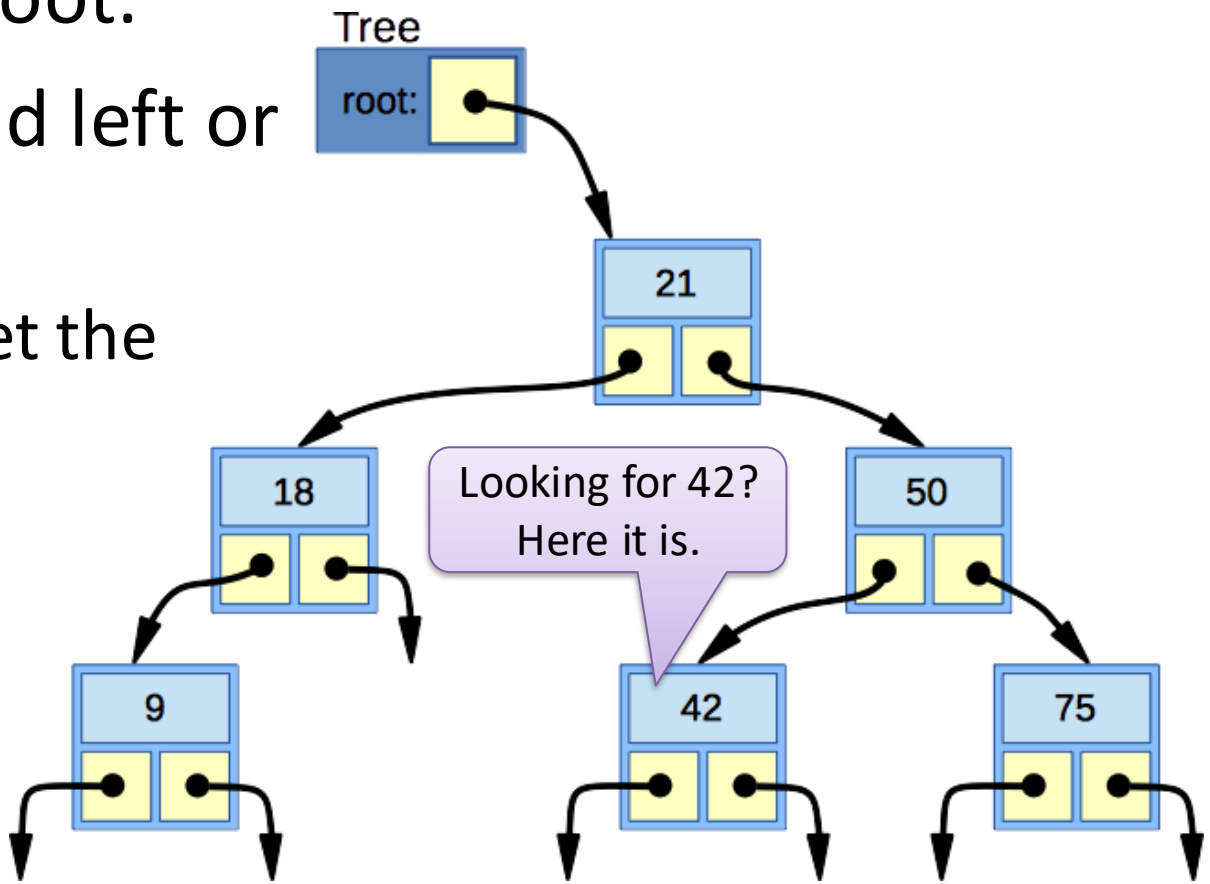
# Finding a Value

- To find a value, you start at the root.
- Then descend left or right.



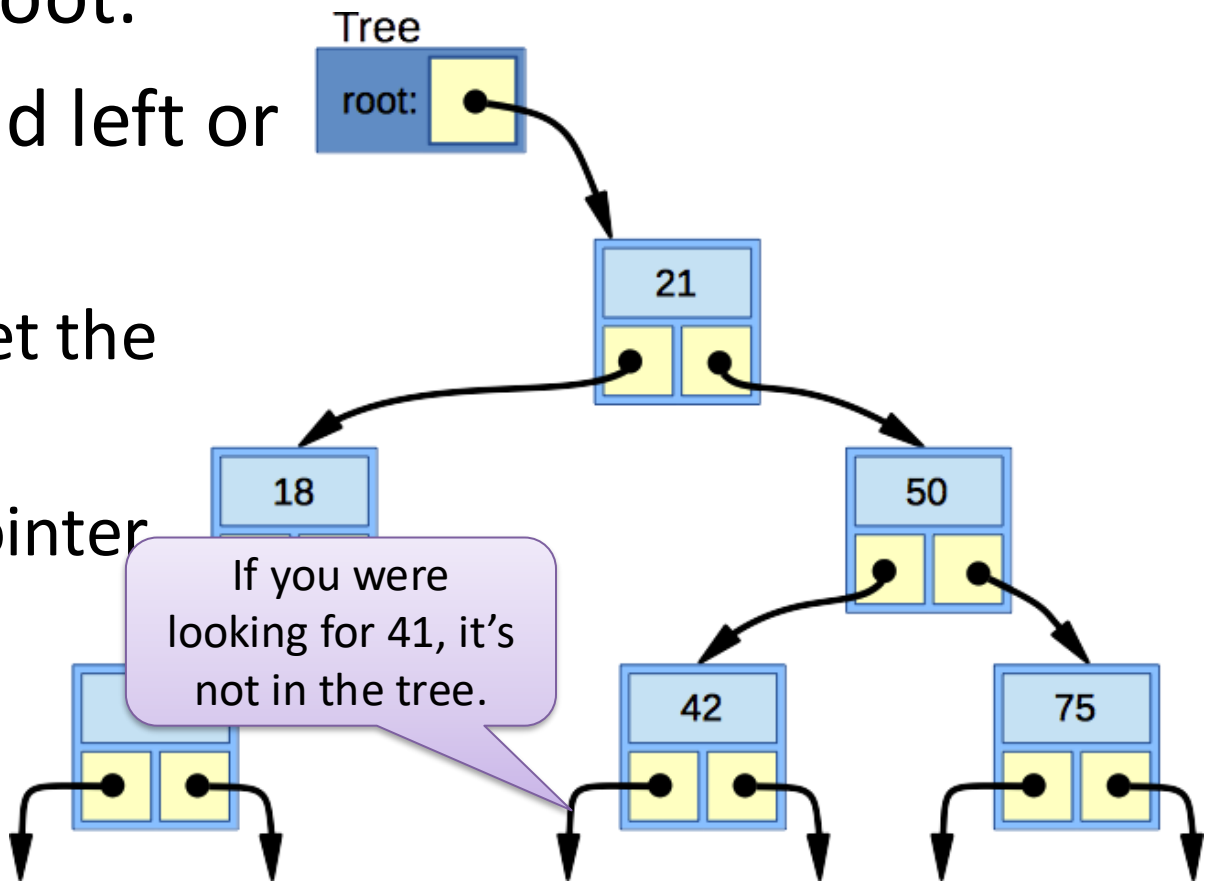
# Finding a Value

- To find a value, you start at the root.
- Then descend left or right.
  - Until you get the value.



# Finding a Value

- To find a value, you start at the root.
- Then descend left or right.
  - Until you get the value.
  - Or a null pointer



# Finding a Value

```
bool containsKey( Tree *tree, int val )
{
  Node *n = tree->root;

  while ( n ) {
    if ( n->key == val )
      return true;

    if ( val < n->key )
      n = n->left;
    else
      n = n->right;
  }

  return false;
}
```

Start at the root.

Descend the tree until we hit a null pointer.

Stop when we find the target value.

Go left or right based on comparison.

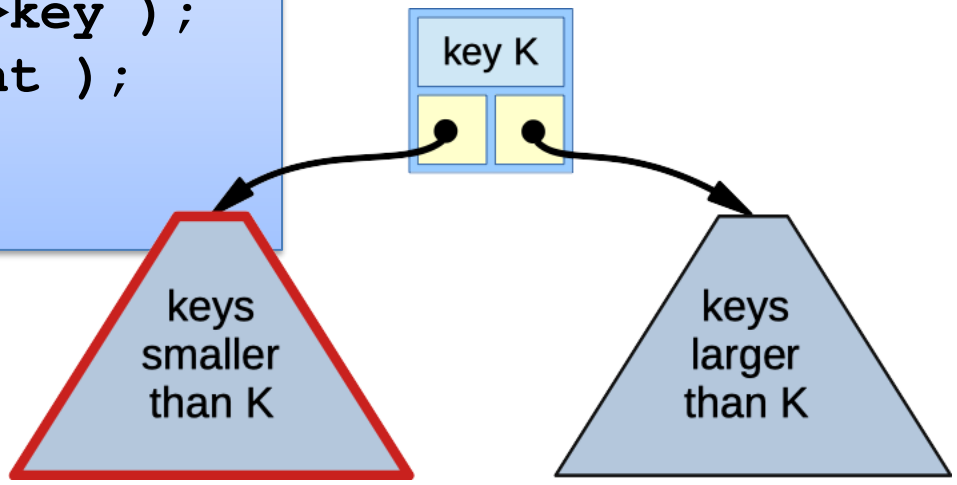
# Tree Traversal

- It's easy to traverse the tree recursively.
- This gives us the keys in sorted order.

```
void printKeys( Node *n )
{
  if ( n ) {
    printKeys( n->left );
    printf( "%d ", n->key );
    printKeys( n->right );
  }
}
```

If the current subtree is non-empty.

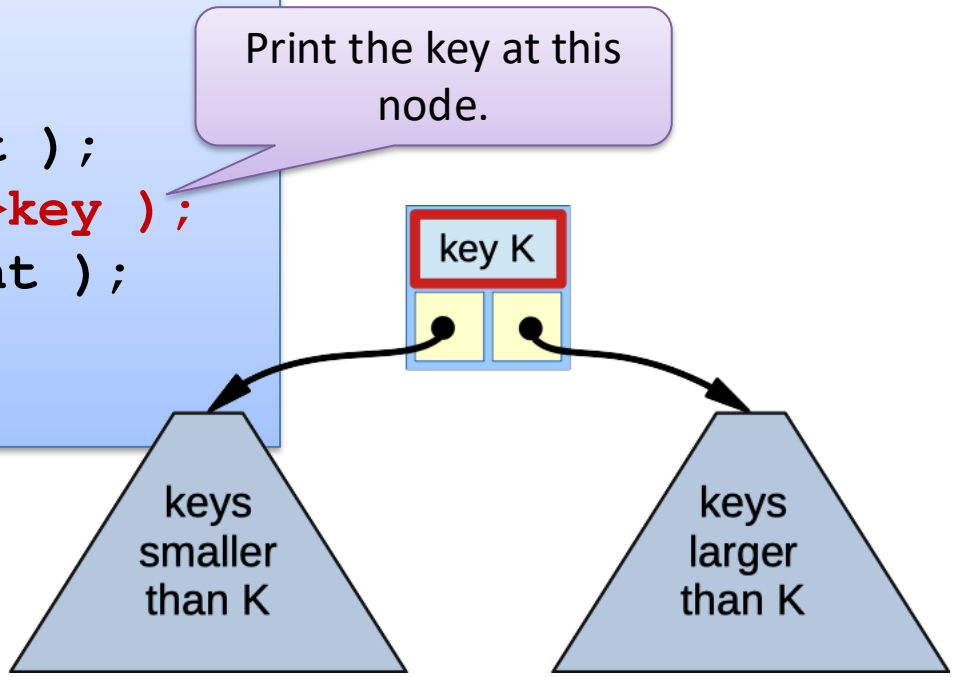
Handle everything in the left subtree.



# Tree Traversal

- It's easy to traverse the tree recursively.
- This gives us the keys in sorted order.

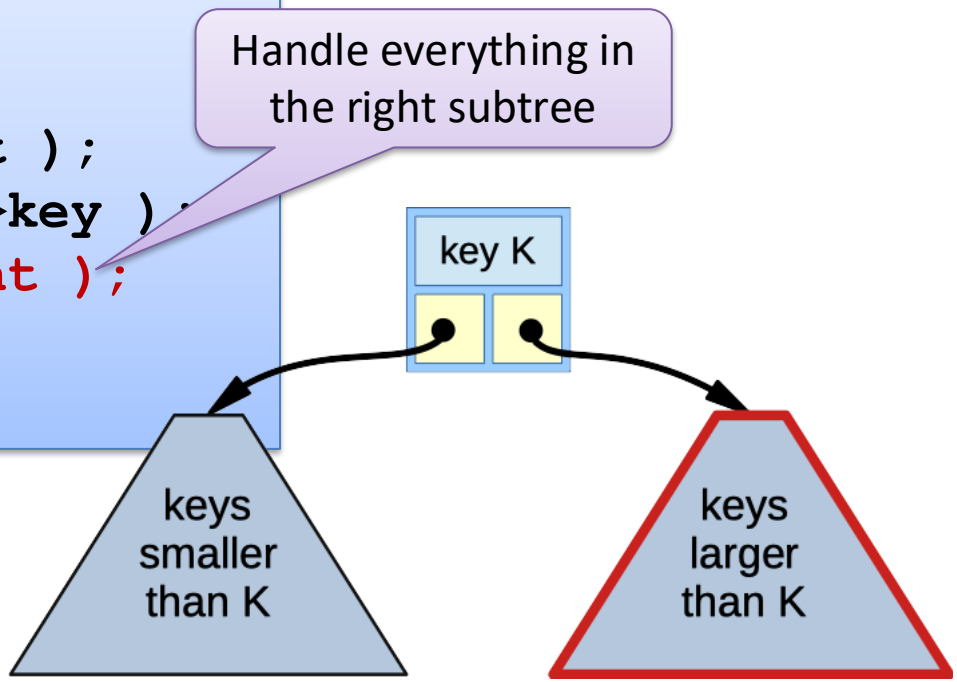
```
void printKeys( Node *n )
{
  if ( n ) {
    printKeys( n->left );
    printf( "%d ", n->key );
    printKeys( n->right );
  }
}
```



# Tree Traversal

- It's easy to traverse the tree recursively.
- This gives us the keys in sorted order.

```
void printKeys( Node *n )
{
  if ( n ) {
    printKeys( n->left );
    printf( "%d ", n->key );
    printKeys( n->right );
  }
}
```



# Generic Tree Traversal

- We can offer a generic traversal interface with a visitor function called on each key.

```
void traverse( Node *n, void (*visit)( int val ) )
{
  if ( n ) {
    traverse( n->left, visit );
    visit( n->key );
    traverse( n->right, visit );
  }
}
```

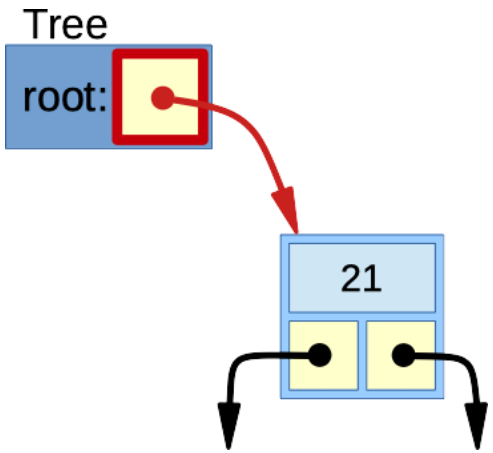
- We can call it like:

```
  traverse( tree.root, myVisitor );
```

Same visitor function as the linked-list example.

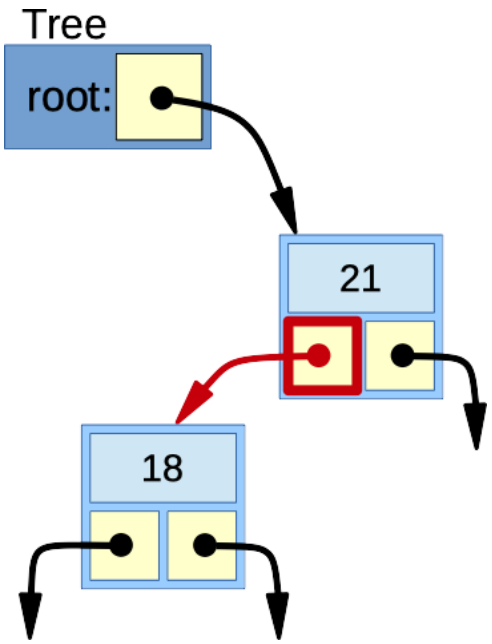
# Inserting Values

- Inserting in a tree requires handling some cases.
  - Maybe the tree is empty, you need a new root node.



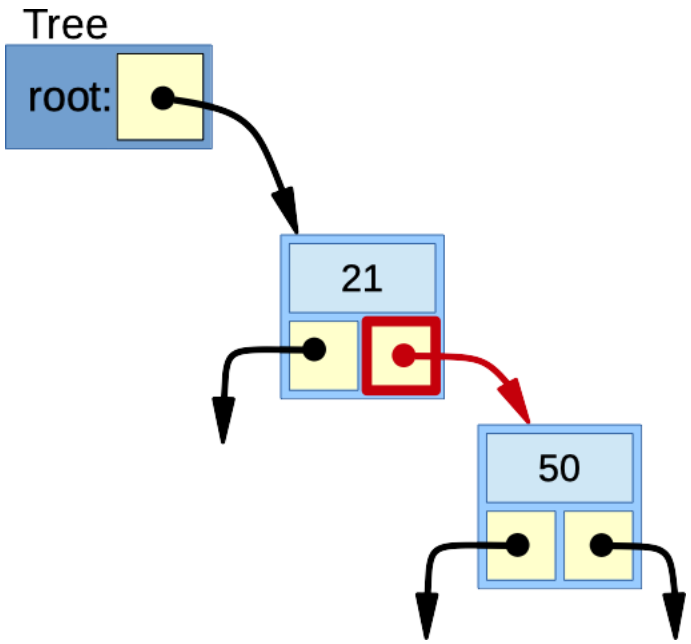
# Inserting Values

- Inserting in a tree requires handling some cases.
  - Maybe the tree is empty, you need a new root node.
  - Maybe the new node needs to be a left child.



# Inserting Values

- Inserting in a tree requires handling some cases.
  - Maybe the tree is empty, you need a new root node.
  - Maybe the new node needs to be a left child.
  - Maybe the new node needs to be a right child.



# Inserting Values

```
bool insertKey( Tree *tree, int val )
{
  if ( ! tree->root ) {
    tree->root = (Node *)malloc( sizeof( Node ) );
    tree->root->key = val;
    tree->root->left = tree->root->right = NULL;
    return true;
  }

  ...
```

No root node.

Make a new node.

Fill in its fields.

# Inserting Values

```
...

Node *parent = tree->root;
while ( parent ) {
  if ( val == parent->key )
    return false;
  if ( val < parent->key ) {
    if ( ! parent->left ) {
      parent->left = (Node *)malloc( sizeof( Node ) );
      *(parent->left) = (Node){ val, NULL, NULL };
      return true;
    } else
      parent = parent->left;
  } else {
    ...
```

Start at the root.

Is this value already in the tree?

Look left or right?

Found a null pointer.

Make a new node and initialize all fields at once.

Go to the left.

# Inserting Values

```
  ...
  } else {
      if ( ! parent->right ) {
        parent->right = (Node *)malloc( sizeof( Node ) );
        *(parent->right) = (Node){ val, NULL, NULL };
        return true;
      } else
        parent = parent->right;
    }
  }

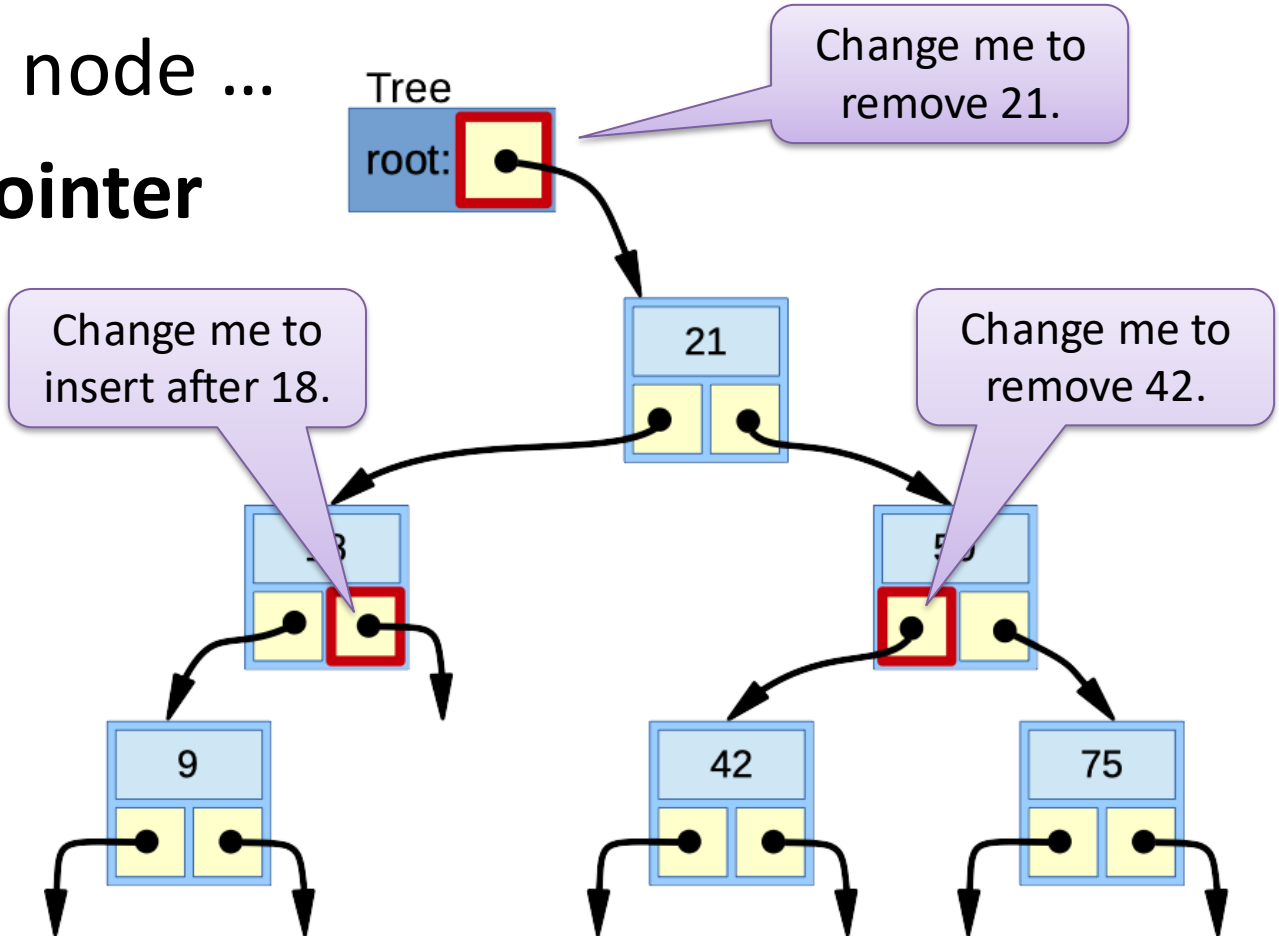
  return false;
}
```

Same code for looking to the right.

Never reached.

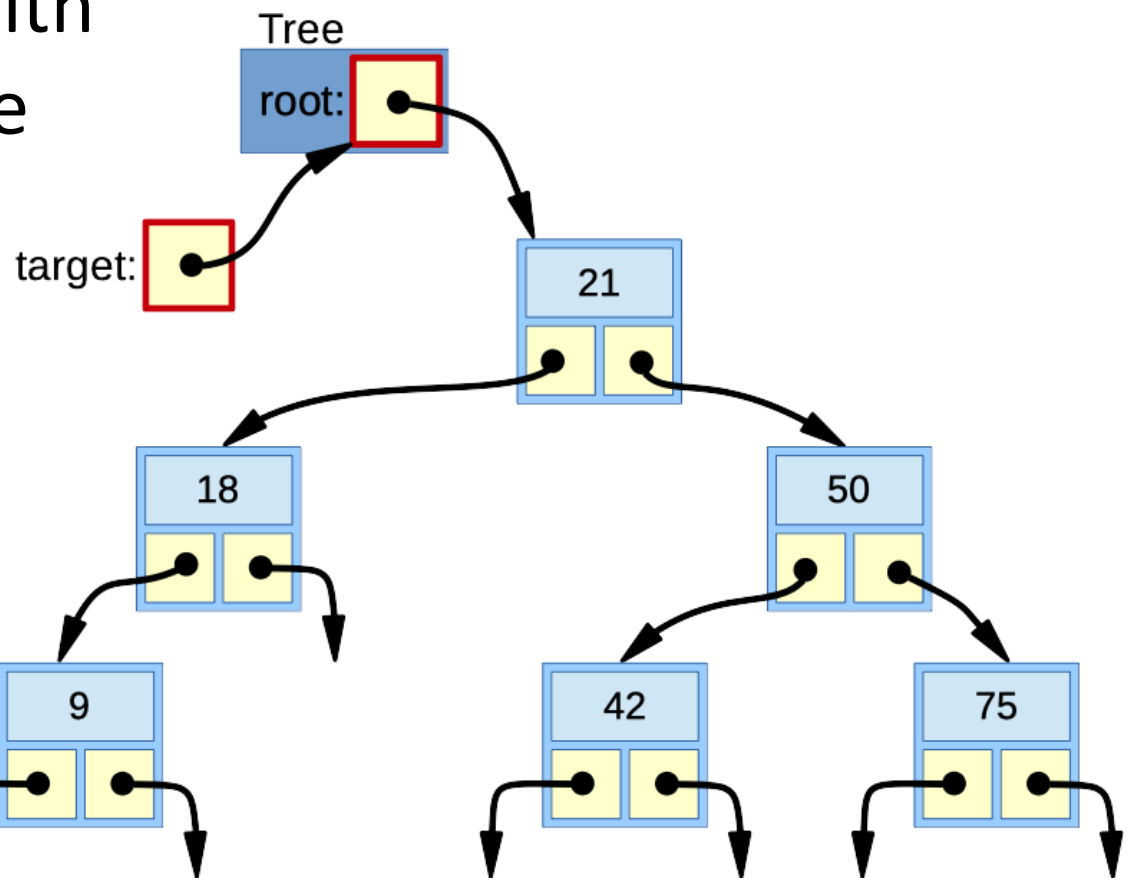
# Finding a Pointer

- Lots of operations require finding …
- … not just a node …
- … but the **pointer** to a node.



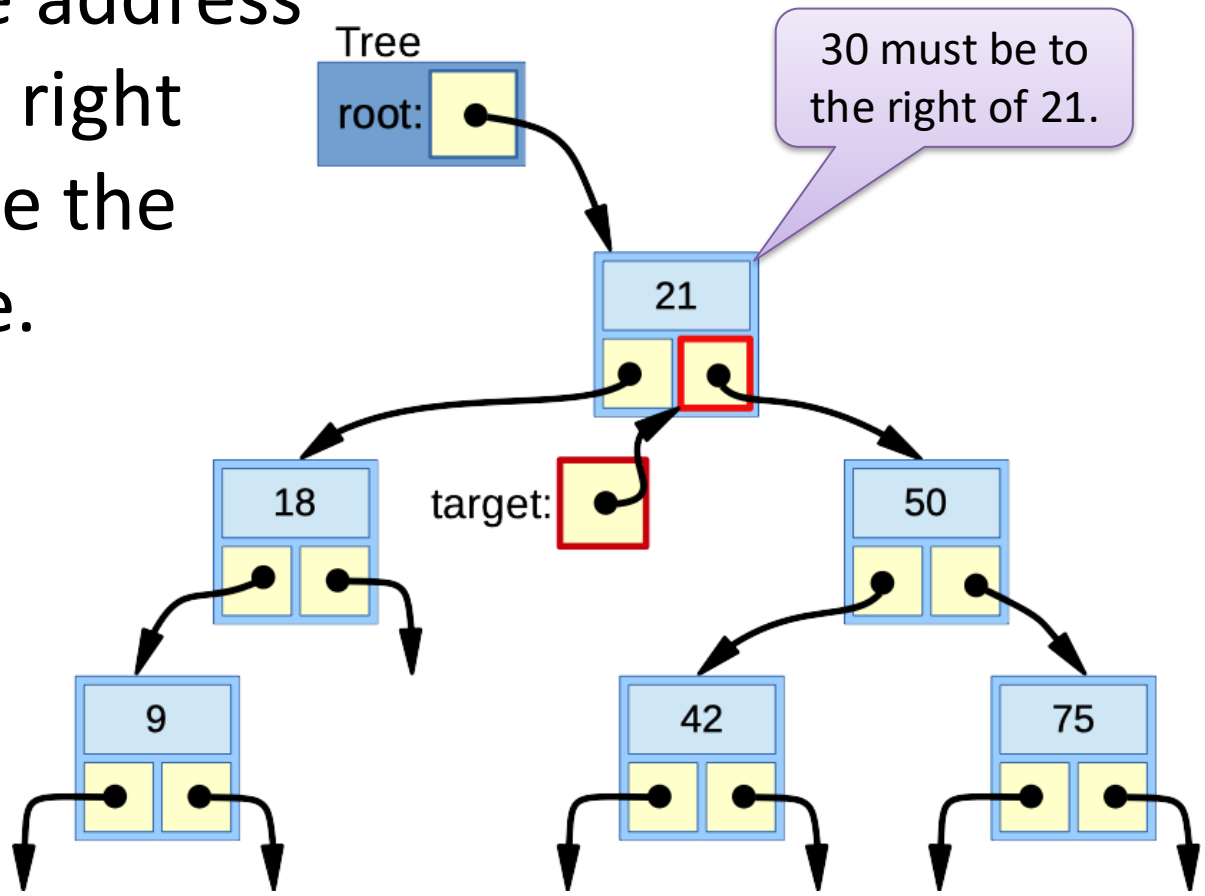
# Finding a Pointer

- Let's say we're looking for the key **30**.
- We can start with a pointer to the root node.



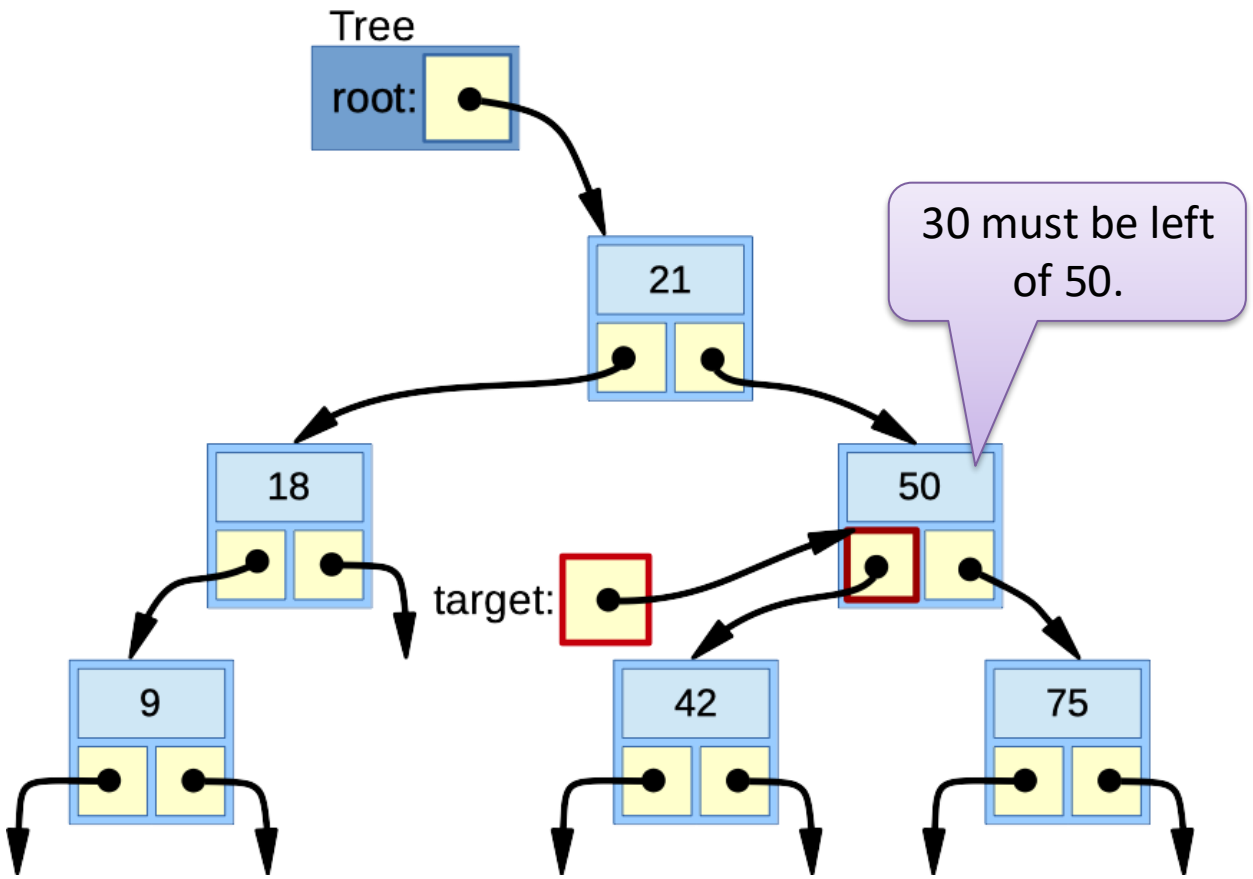
# Finding a Pointer

- Descending the tree by taking the address of the left or right pointer inside the current node.



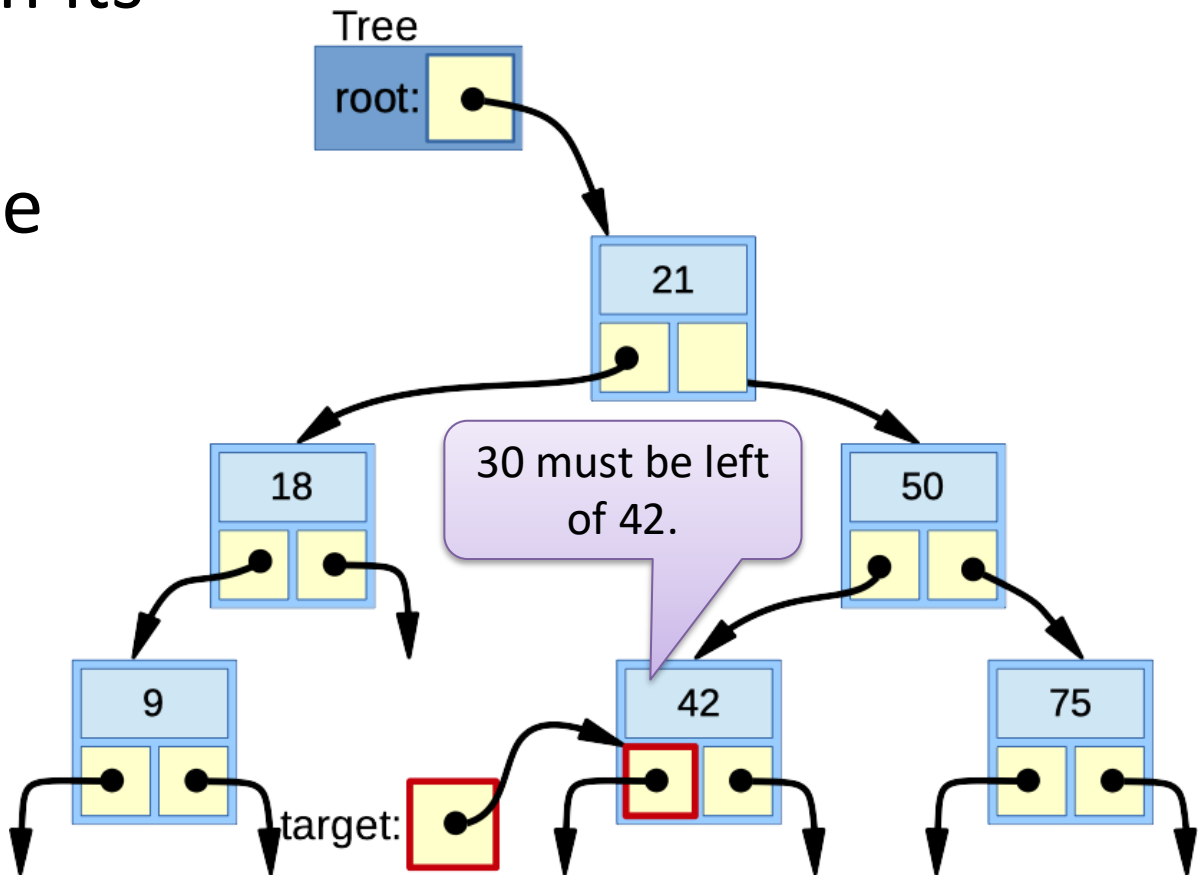
# Finding a Pointer

- And so on.



# Finding a Pointer

- If we reach a null … we can return its address.
- That might be useful.



# Finding a Pointer

```
Node **findPointer( Tree *tree, int val )
{
  Node **target = &( tree->root );
  while ( *target && (*target)->key != val )
    if ( val < (*target)->key )
      target = &(*target)->left;
    else
      target = &(*target)->right;

  return target;
}
```

Start at the address of the root.

Stop when you find val or a null.

Take the address of the left or right pointer.

# Finding a Value, Simplified

- Finding a value is trivial, if we use findPointer() to do most of the work.

```
bool containsKey( Tree *tree, int val )
{
  Node **target = findPointer( tree, val );
  return *target;
}
```

Return true if target points to a non-null.

# Insert, Simplified

- Insert is simple, if we let findPointer() locate the pointer we need to change.

```
bool insertKey( Tree *tree, int val )
{
  Node **target = findPointer( tree, val );

  if ( *target )
    return false;

  *target = (Node *)malloc( sizeof( Node ) );
  **target = (Node){ val, NULL, NULL };
  return true;
}
```

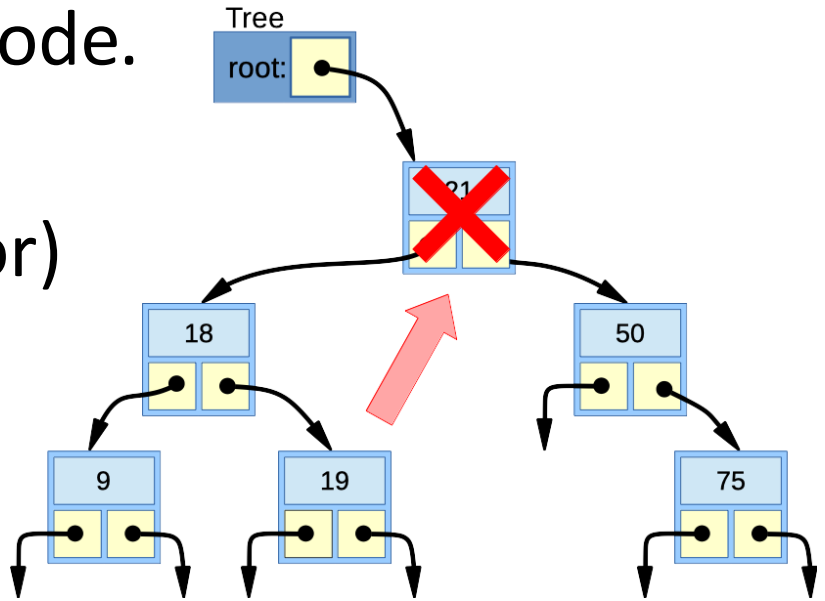
Find where this value would be.

Is the value already there?

Make this pointer point to a new leaf node with the given key.

# Tree Removal

- If we have time, we'll look at removal from a tree.
- This is more complicated … You can't just remove a node.
- You need to find another node (e.g., its predecessor) to replace it with.
- Here, using a reference to a pointer can let you fold a lot of cases together.



# Tree Removal

```
bool removeKey( Tree *tree, int val )
{
  if ( tree->root && tree->root->key == val ) {
    Node *target = tree->root;
    tree->root = findReplacement( target );
    free( target );
    return true;
  }

  Node *parent = tree->root;

  while ( parent ) {
    if ( val < parent->key ) {
      if ( parent->left && parent->left->key == val ) {
        Node *target = parent->left;
        parent->left = findReplacement( target );
        free( target );
        return true;
      } else
        parent = parent->left;
    ...
```

Are we removing the root?

Helper to get us a replacement node.

Look within the tree.

Handle removing a left child.

# Tree Removal

```
    ...
 } else {
      if ( parent->right && parent->right->key == val ) {
        Node *target = parent->right;
        parent->right = findReplacement( target );
        free( target );
        return true;
      } else
        parent = parent->right;
    }
  }

  return false;
}
```

Handle removing a right child.

# Extracting a Replacement

```
static Node *findReplacement( Node *n )
{
  if ( ! n->left )
    return n->right;

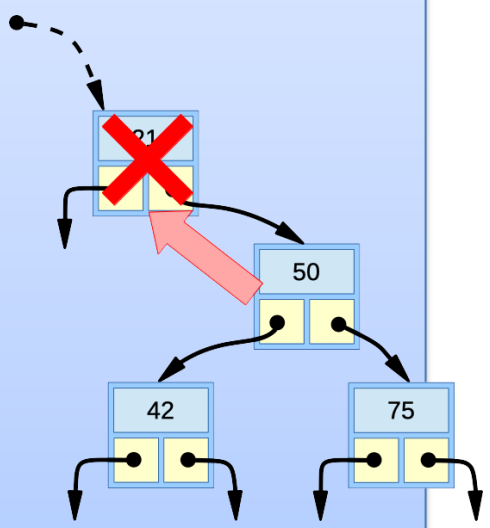
  if ( ! n->left->right ) {
    n->left->right = n->right;
    return n->left;
  }

  Node *par = n->left;
  while ( par->right->right )
    par = par->right;

  Node *pred = par->right;
  par->right = pred->left;

  pred->left = n->left;
  pred->right = n->right;
  return pred;
}
```

No left child, replace with the right child.



# Extracting a Replacement

```
static Node *findReplacement( Node *n )
{
  if ( ! n->left )
    return n->right;

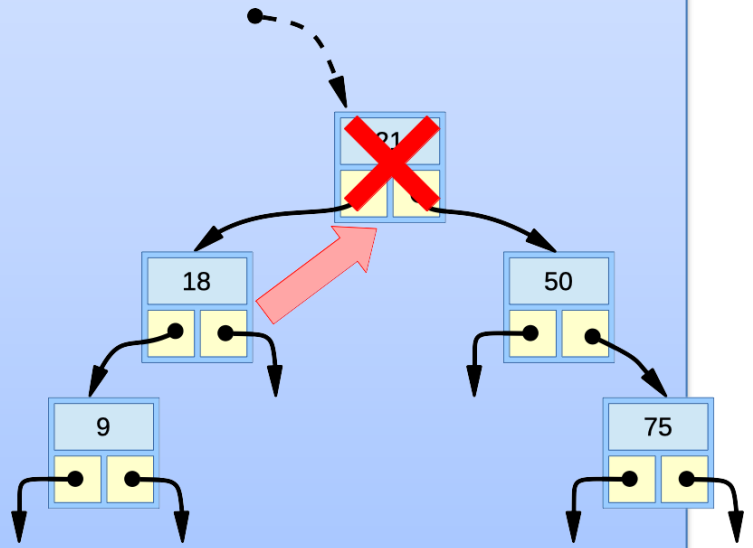
  if ( ! n->left->right ) {
    n->left->right = n->right;
    return n->left;
  }

  Node *par = n->left;
  while ( par->right->right )
    par = par->right;

  Node *pred = par->right;
  par->right = pred->left;

  pred->left = n->left;
  pred->right = n->right;
  return pred;
}
```

Left child has no right child, replace with that.



# Extracting a Replacement

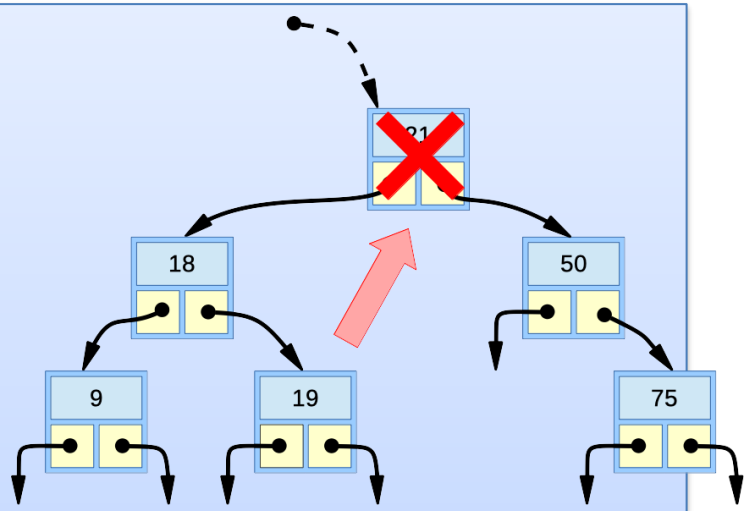
```
static Node *findReplacement( Node *n )
{
  if ( ! n->left )
    return n->right;

  if ( ! n->left->right ) {
    n->left->right = n->right;
    return n->left;
  }

  Node *par = n->left;
  while ( par->right->right )
    par = par->right;

  Node *pred = par->right;
  par->right = pred->left;

  pred->left = n->left;
  pred->right = n->right;
  return pred;
}
```




Find rightmost descendant.

Remove it from its parent.

Prepare that node to replace n.

# Tree Removal, Simplified

- Removal is easier, once we find the right pointer to change.

```
bool removeKey( Tree *tree, int val )
{
  Node **target = findPointer( tree, val );

  if ( ! *target )
    return false;

  Node *n = *target;
  *target = findReplacement( n );
  free( n );
  return true;
}
```

Find the pointer we need to change.

Is there a node there?

Replace it.

# Extracting a Replacement, Simplified

- Pointer-to-pointer helps in findReplacement()

```
static Node *findReplacement( Node *n )
{
  if ( ! n->left )
    return n->right;

  Node **predPtr = &( n->left );
  while ( (*predPtr)->right )
    predPtr = &(*predPtr)->right;

  Node *pred = *predPtr;
  *predPtr = pred->left;

  pred->left = n->left;
  pred->right = n->right;
  return pred;
}
```

No left child, replace with the right child.

Find pointer to our predecessor.

Remove it.

Prepare that node to replace n.